



Elaborado por:
Micaele Dias Alves
Data: 13/06/2026

Aprovado por:
Fernando Albuquerque
Revisão: 0

CADERNO DE ARQUITETURA

1. Introdução



Este documento descreve a filosofia, as decisões, as restrições, as justificativas, os elementos significativos e demais aspectos gerais do sistema de gerenciamento de projetos baseado no método Kanban que orientam o design e a implementação da solução. Seu objetivo é registrar as principais escolhas arquiteturais realizadas pela equipe, servindo como referência para o desenvolvimento, manutenção e evolução do sistema. Este documento complementa os demais artefatos do projeto, em especial o Documento de Visão e Escopo e o Projeto Físico do Banco de Dados.

2. Objetivos e Filosofia Arquitetural



A arquitetura do sistema foi concebida com base em princípios de simplicidade, modularidade e facilidade de manutenção, considerando que a solução destina-se a equipes acadêmicas e pequenos grupos de desenvolvimento de software. As principais diretrizes filosóficas que orientam a arquitetura são:

- Separação de responsabilidades: a solução é estruturada em camadas distintas que separam a interface com o usuário, as regras de negócio e a persistência dos dados, facilitando a manutenção e a evolução independente de cada componente.
- Simplicidade e objetividade: a arquitetura evita complexidade desnecessária, adotando padrões bem estabelecidos e tecnologias amplamente conhecidas, reduzindo a curva de aprendizado da equipe.
- Rastreabilidade e transparência: o sistema deve ser capaz de registrar e exibir o estado das atividades em tempo real, garantindo visibilidade do fluxo de trabalho conforme os princípios do método Kanban.
- Disponibilidade e acessibilidade web: a solução deve funcionar em ambiente web, acessível por navegadores modernos, sem dependência de instalação local de software pelo usuário.
- Escalabilidade controlada: embora o foco seja equipes pequenas, a arquitetura não deve criar barreiras para uma eventual expansão do número de usuários ou projetos.

2.1. Questões que orientam a filosofia



As decisões arquiteturais foram influenciadas pelos seguintes fatores: (a) necessidade de suportar múltiplos usuários acessando simultaneamente o mesmo quadro Kanban; (b) requisito de autenticação e controle de acesso por projeto; (c)

cálculo e exibição de métricas em tempo de execução (Cycle Time, Lead Time, Throughput, WIP); (d) persistência confiável das informações de projetos, quadros, raias e cartões; e (e) interface intuitiva com atualização visual do estado dos cartões sem necessidade de recarregamento completo da página.

3. Suposições e Dependências



As suposições e dependências listadas a seguir influenciam diretamente as decisões arquiteturais adotadas. Alterações nesses fatores podem exigir revisão da arquitetura proposta.

Suposição / Dependência	Descrição
Disponibilidade de infraestrutura	Assume-se que o ambiente de produção disponibilizará servidor web, servidor de banco de dados e conectividade à internet estável para hospedagem da aplicação.
Acesso via navegador web	Os usuários acessarão o sistema por meio de navegadores web modernos (Chrome, Firefox, Edge, Safari), sem necessidade de cliente nativo. Uma interface de linha de comando (CLI) também está disponível para uso em ambiente de desenvolvimento e administração local.
Autenticação por credenciais próprias	O sistema gerenciará cadastro e autenticação de usuários internamente, sem integração com provedores externos de identidade (ex.: OAuth, SSO) nesta versão.
Banco de dados PostgreSQL	A persistência de dados depende do SGBD PostgreSQL, cuja disponibilidade é pressuposta no ambiente de implantação
Gestão de credenciais por variáveis de ambiente	As credenciais de acesso ao banco de dados (host, nome do banco, usuário e senha) devem ser configuradas via variáveis de ambiente ou arquivo de configuração separado (.env), nunca hardcodadas diretamente no código-fonte, especialmente em repositórios públicos.
Sem integrações externas	Esta versão do sistema não depende de integração com plataformas externas de gerenciamento de projetos ou serviços de notificação. O GitHub é utilizado pela equipe para controle de versões dos artefatos, mas não é integrado funcionalmente ao sistema em produção.
Equipe com conhecimento técnico básico	Assume-se que os desenvolvedores envolvidos possuem familiaridade com desenvolvimento web, SQL e conceitos de arquitetura em camadas.
Ambiente acadêmico/pequenas equipes	O sistema é dimensionado para equipes de até dezenas de usuários simultâneos, sem requisitos de alta disponibilidade ou escalabilidade massiva nesta versão.

4. Requisitos Arquiteturalmente Significativos



Os requisitos listados a seguir possuem impacto direto nas decisões arquiteturais do sistema. Foram derivados das histórias de usuário e dos requisitos gerais definidos no Documento de Visão e Escopo.

ID	História de Usuário / Requisito	Impacto Arquitetural
US-ACC-01	Cadastro de usuário	Necessidade de camada de autenticação e hash seguro de senhas (bcrypt ou similar).
US-ACC-02	Autenticação no sistema	Implementação de sessão ou token JWT para controle de acesso às rotas protegidas.
US-ACC-03	Participação em múltiplos projetos	Tabela de junção USUARIO_QUADRO com controle de papéis (DONO, EDITOR, LEITOR).
US-BRD-01/04	CRUD de quadros Kanban	Entidade QUADRO persistida no banco com operações completas na camada de negócio.
US-SWI-01	Gerenciamento de raias (swimlanes)	Entidade RAIA vinculada a QUADRO, com suporte a WIP (QTD_MAX) por coluna
US-CRD-01/04	CRUD de cartões de atividades	Entidade CARTAO com relacionamentos a RAIA, COR e USUARIO via tabelas de junção.
US-FLW-01/02	Movimentação de cartões entre colunas e raias	Lógica de negócio para validação de WIP antes de movimentar cartões.
US-FLW-03	Controle de WIP por coluna	Restrição lógica e visual impedindo inserção de cartões além do limite configurado.
US-MET-01/04	Cálculo de Cycle Time, Lead Time, Throughput e WIP	Camada de serviço dedicada ao cálculo de métricas com base nas datas dos cartões.

5. Decisões, Restrições e Justificativas



A tabela a seguir consolida as principais decisões arquiteturais tomadas pela equipe, bem como as restrições identificadas e as justificativas que fundamentam cada escolha.

Decisão / Restrição	Justificativa
Arquitetura em camadas (Layered Architecture)	A separação em camadas de Apresentação, Negócio e Persistência facilita a manutenção, o teste independente de componentes e a evolução do sistema sem impactos cruzados entre camadas.
Aplicação web (interface via navegador)	Elimina a necessidade de instalação de cliente local, facilita o acesso por diferentes dispositivos e sistemas operacionais, e alinha-se ao requisito de compatibilidade com ambiente web.
Banco de dados relacional PostgreSQL	PostgreSQL oferece suporte robusto a transações, integridade referencial, tipos de dados avançados e é amplamente utilizado em ambientes de produção. A modelagem relacional é adequada às entidades do sistema (Usuário, Quadro, Raia, Cartão).
Estrutura modular da aplicação	A organização modular promove a reutilização de componentes, facilita o trabalho paralelo da equipe de desenvolvimento e reduz o acoplamento entre funcionalidades distintas.
Autenticação por credenciais próprias (sem OAuth externo)	Reduz dependências externas e simplifica a implementação para o contexto acadêmico, sem comprometer a segurança básica com uso de hash de senhas.

Sem integração com sistemas externos nesta versão	Decisão de escopo que reduz a complexidade da implementação inicial, conforme definido no Documento de Visão e Escopo. Integrações poderão ser adicionadas em versões futuras.
Controle de WIP implementado na camada de negócio	Centralizar a validação de limites WIP na camada de negócio garante que a regra seja aplicada independentemente do canal de acesso (interface web ou API), evitando inconsistências.
Cálculo de métricas na camada de serviço	Isolar o cálculo de Cycle Time, Lead Time, Throughput e WIP em serviços dedicados facilita testes unitários e permite futuras otimizações sem impacto na interface.

6. Mecanismos Arquiteturais



Os mecanismos arquiteturais são soluções recorrentes adotadas para tratar aspectos transversais do sistema. A tabela a seguir descreve cada mecanismo identificado, seu propósito e seu estado atual no desenvolvimento.

Mecanismo	Descrição
Autenticação e Controle de Acesso	Mecanismo responsável por verificar a identidade do usuário e controlar o acesso às funcionalidades do sistema. Utiliza sessão ou token (JWT) após validação de credenciais com senha armazenada em hash.
Persistência de Dados	Camada de acesso ao banco de dados PostgreSQL, responsável pelo mapeamento entre objetos da aplicação e tabelas relacionais. Abstrai operações de CRUD e garante integridade transacional.
Controle de WIP	Mecanismo de negócio que valida, antes de qualquer movimentação de cartão, se o limite máximo de cartões por coluna (QTD_MAX) foi atingido, impedindo inserções além do limite configurado.
Cálculo de Métricas Kanban	Serviço responsável pelo cálculo de Cycle Time (tempo em execução), Lead Time (tempo total), Throughput (entregas por período) e WIP (cartões em andamento), com base nas datas registradas nos cartões.
Gerenciamento de Sessão	Controle do estado de autenticação do usuário durante a sessão ativa, garantindo que apenas usuários autenticados acessem as rotas protegidas da aplicação.
Tratamento de Erros	Estratégia centralizada de captura e resposta a erros da aplicação, incluindo validações de entrada, falhas de banco de dados e violações de regras de negócio, com mensagens amigáveis ao usuário.

7. Principais Abstrações



As abstrações a seguir representam os conceitos centrais do domínio do sistema, correspondendo às entidades e classes de análise que estruturam o modelo da solução.

Abstração	Descrição
Usuário	Representa qualquer pessoa autenticada no sistema. Possui credenciais de acesso, perfil e pode participar de um ou mais projetos com diferentes níveis de permissão (Dono, Editor, Leitor).
Quadro Kanban	Elemento central do sistema que organiza as atividades de um projeto. Contém raias (colunas) que representam os estágios do fluxo de trabalho e está associado a um ou mais usuários.
Raia / Coluna	Representa um estágio do fluxo de trabalho dentro de um quadro Kanban (ex.: A Fazer, Fazendo, Feito). Cada raia possui um limite máximo de cartões (WIP - QTD_MAX) e um nome descritivo.
Cartão de Atividade	Unidade básica de trabalho do sistema, representando uma tarefa ou atividade. Contém identificador, nome, descrição, datas de início e fim, posição na raia (ORDEM), referência de cor para categorização visual (ID_COR) e associação a usuários responsáveis via tabela USUARIO_CARTAO.
Cor	Atributo visual associado a cartões para categorização e identificação rápida de tipos ou prioridades de atividades por meio de código de cor ou rótulo semântico.
Associação Usuário-Cartão (Usuario_Cartao)	Relacionamento que vincula usuários a cartões específicos, definindo o tipo de acesso (Dono, Editor, Leitor) e a data de associação.
Associação Usuário-Quadro (Usuario_Quadro)	Relacionamento que vincula usuários a quadros Kanban, controlando participação em projetos e nível de permissão sobre o quadro.
Métricas Kanban	Conjunto de indicadores calculados a partir dos dados dos cartões: Cycle Time (tempo entre início e conclusão), Lead Time (tempo total desde a criação), Throughput (volume de entregas) e WIP (itens em andamento).

8. Camadas e Framework Arquitetural



O sistema adota o padrão arquitetural em camadas (Layered Architecture), com separação clara entre as responsabilidades de apresentação, regras de negócio e persistência dos dados.

8.1. Camada de Apresentação (Interface com o Usuário)



Responsável pela interação direta com o usuário. Implementada como aplicação web acessível por navegadores, esta camada exibe os quadros Kanban, os cartões de atividades e as métricas, além de capturar as ações do usuário (criar, mover, editar cartões). Deve ser intuitiva, responsiva e refletir visualmente o estado atualizado do fluxo de trabalho.

8.2. Camada de Negócio (Lógica da Aplicação)



Concentra as regras de negócio do sistema, incluindo: validação de limites WIP antes da movimentação de cartões; cálculo das métricas Kanban (Cycle Time, Lead Time,

Throughput e WIP); controle de acesso por nível de permissão (Dono, Editor, Leitor); e coordenação das operações de CRUD sobre quadros, raia e cartões. Esta camada é isolada das demais, permitindo sua evolução independente.

8.3. Camada de Persistência (Banco de Dados)



Responsável pelo armazenamento e recuperação das informações do sistema por meio do SGBD PostgreSQL. Gerencia as operações sobre as tabelas USUARIO, QUADRO, RAIA, CARTAO, COR, USUARIO_CARTAO e USUARIO_QUADRO, garantindo integridade referencial, consistência transacional e aplicação das restrições definidas no Projeto Físico do Banco de Dados.

8.4. Impacto das Ferramentas e Frameworks



A escolha das tecnologias e ferramentas utilizadas impacta diretamente a arquitetura do sistema. A seguir são descritos os principais impactos identificados:

Ferramenta / Framework	Impacto Arquitetural
Python + Flask (Back-end)	Framework web minimalista utilizado na camada de apresentação e negócio. O Flask roteia as requisições HTTP, serve os templates e coordena as chamadas aos serviços. Sua natureza leve é adequada ao escopo do sistema e não impõe estrutura rígida de organização.
psycopg2 (Adaptador PostgreSQL)	Biblioteca Python utilizada para conexão e execução de queries SQL nativas no PostgreSQL, sem uso de ORM. Isso implica mapeamento manual entre os objetos Python e os registros do banco, com controle total sobre as queries executadas.
PostgreSQL (SGBD)	Define a estrutura relacional da camada de persistência, com suporte a transações ACID, chaves estrangeiras com ON DELETE CASCADE/SET NULL e constraints de integridade. O código organiza o acesso ao banco no módulo db/.
Templates Jinja2 (Apresentação)	Camada de apresentação implementada via templates HTML renderizados pelo Flask, organizados no diretório templates/. Reflete a separação física entre apresentação, negócio (services/) e persistência (db/).
draw.io (Modelagem)	Utilizado para criação dos diagramas de entidade-relacionamento (MER/DER). Não impacta a arquitetura em produção, mas documenta as decisões de estrutura de dados.
Taiga.io (Gerenciamento)	Plataforma utilizada para o gerenciamento ágil do projeto com quadro Kanban. Não integrada ao sistema em desenvolvimento, sendo utilizada apenas para organização e acompanhamento da equipe.
GitHub (Controle de Versões)	Centraliza o armazenamento e versionamento dos artefatos e do código-fonte do protótipo. Impacta o fluxo de trabalho da equipe, mas não a arquitetura interna do sistema.

9. Visões Arquiteturais



As visões arquiteturais a seguir descrevem o sistema sob diferentes perspectivas, permitindo que diferentes partes interessadas compreendam aspectos específicos da solução.

9.1. Visão Lógica



A visão lógica descreve a estrutura e o comportamento das partes arquiteturalmente significativas do sistema. O sistema é organizado nos seguintes módulos principais:

- Módulo de Autenticação: gerência cadastro, login e controle de sessão dos usuários.
- Módulo de Projetos e Quadros: responsável pela criação e gerenciamento de quadros Kanban e suas raias.
- Módulo de Cartões: gerencia o ciclo de vida completo dos cartões de atividades, incluindo criação, edição, movimentação e exclusão.
- Módulo de Métricas: calcula e exibe os indicadores Cycle Time, Lead Time, Throughput e WIP com base nos dados dos cartões.
- Módulo de Persistência: abstrai o acesso ao banco de dados PostgreSQL para os demais módulos.

As entidades persistentes que suportam esta visão são: USUARIO, QUADRO, RAIA, CARTAO, COR, USUARIO_CARTAO e USUARIO_QUADRO, conforme detalhado no Projeto Físico do Banco de Dados.

9.2. Visão Operacional



A visão operacional descreve os nós físicos do sistema e os processos que neles executam. O sistema é composto pelos seguintes elementos de infraestrutura:

- Servidor de Aplicação Web: hospeda a camada de apresentação e a camada de negócio da aplicação, responsável por processar as requisições dos usuários e servir as páginas/respostas via Flask.
- Servidor de Banco de Dados: executa o SGBD PostgreSQL, armazenando e gerenciando todos os dados do sistema.
- Cliente (Navegador Web): dispositivo do usuário que acessa a aplicação via navegador web compatível, sem instalação de software adicional. É o canal de acesso principal da interface gráfica.
- Interface de Linha de Comando (CLI): canal de acesso alternativo disponibilizado pelo módulo run.py, que oferece um menu interativo em terminal cobrindo as operações principais do sistema (listar quadros, criar raias, criar cartões, entre outras). Destinado a uso em ambiente de desenvolvimento e administração local.

A comunicação entre o cliente web e o servidor de aplicação ocorre via protocolo HTTP/HTTPS. A comunicação entre o servidor de aplicação e o banco de dados ocorre via protocolo nativo do PostgreSQL por meio da biblioteca psycopg2, em rede interna do ambiente de hospedagem.

9.3. Visão de Casos de Uso



Os casos de uso arquiteturalmente significativos que guiam a estrutura do sistema são:

- UC-01: Autenticar no sistema — influencia a estrutura de controle de acesso e sessão.
- UC-02: Gerenciar quadros Kanban — central para a organização dos módulos de quadros e raias.

- UC-03: Criar e movimentar cartões — define o fluxo principal de operação e a lógica de validação de WIP.
- UC-04: Calcular e visualizar métricas — determina a necessidade da camada de serviço de métricas separada.
- UC-05: Participar de múltiplos projetos — fundamenta a estrutura de controle de acesso por nível de permissão.

Aprovação e autorização de conclusão de documento

Equipe responsável	Matrícula	Data de revisão
Gabriel Pessoa Faustino	231006121	
Karina Escalona Escalona	231006140	
Leticia Gonçalves Bonfim	241002411	
Maria Vidal Macedo	232013238	
Micaele Dias Alves	231021450	